

Analiza efektywności wykonania programów równoległych i sekwencyjnych na podstawie algorytmu Bubble Sort

Twórcy:

- Paweł Czernice – Kod programu równoległego, wykresy przyśpieszenia.
- Yevhenii Kiliarov – Kod programu sekwencyjnego, tabela.
- Grzegorz Bzymek – Prezentacja, wnioski.

Spis treści:

Kod programu sekwencyjnego dla małej ilości danych.....	2
Kompilacja programu sekwencyjnego dla małej ilości danych.....	3
Kod programu sekwencyjnego dla dużej ilości danych.....	3
Kompilacja programu sekwencyjnego dla dużej ilości danych.....	4
Czas wykonania kodu sekwencyjnego.....	5
Kod programu równoległego dla małej ilości danych.....	7
Kompilacja programu równoległego dla małej ilości danych.....	7
Kod programu równoległego dla dużej ilości danych.....	8
Kompilacja programu równoległego dla dużej ilości danych.....	9
Użyte elementy OpenMP.....	9
Przykładowy skrypt kolejkowy.....	10
Specyfikacja komputera.....	11
Tabele.....	12
Wykresy:	
• Wykres 1.....	12
• Wykres 2.....	13
Wnioski.....	13

Kod programu sekwencyjnego dla małej ilości danych:

```
#include #include #include <omp.h>
```

```
using namespace std;
```

```

void swapElement(float* firstNum, float* secondNum){ float temp = *firstNum; *firstNum =
*secondNum; *secondNum = temp; }

void bubbleSort(float* arr, int size){ for (int i = 0; i < size - 1; i++){ for (int j = 0; j < size - i - 1;
j++){ if (arr[j] > arr[j+1]){ swapElement(&arr[j], &arr[j+1]); } }

}

}

void fillWithRandomNumbers(float* arr, int size){ for (int i = 0; i < size; i++){ arr[i] = static_cast
(rand()) / static_cast (size); } }

int main(){ int size = 5000; float* arr = new float[size];

    fillWithRandomNumbers(arr, size);

    double start = omp_get_wtime();

    bubbleSort(arr, size);

    double end = omp_get_wtime();

    cout << "Time: " << (end - start) / 60 << " min\n";

    delete[] arr;

    return 0;

}

```

Sposób kompilacji:

Nazwa kompilatora: g++

Wersja kompilatora: gcc version 8.5.0 20210514 (Red Hat 8.5.0-28) (GCC)

Kompilacja programu: g++ -fopenmp sequenceN5000.cpp -o sequence

Kod programu sekwencyjnego dla dużej ilości danych:

```
#include #include #include <omp.h>
```

```

using namespace std;

void swapElement(float* firstNum, float* secondNum){ float temp = *firstNum; *firstNum =
*secondNum; *secondNum = temp; }

void bubbleSort(float* arr, int size){ for (int i = 0; i < size - 1; i++){ for (int j = 0; j < size - i - 1;
j++){ if (arr[j] > arr[j+1]){ swapElement(&arr[j], &arr[j+1]); } }
}

}

void fillWithRandomNumbers(float* arr, int size){ for (int i = 0; i < size; i++){ arr[i] = static_cast
(rand()) / static_cast (size); } }

int main(){ int size = 350000; float* arr = new float[size];

    fillWithRandomNumbers(arr, size);

    double start = omp_get_wtime();

    bubbleSort(arr, size);

    double end = omp_get_wtime();

    cout << "Time: " << (end - start) / 60 << " min\n";

    delete[] arr;

    return 0;

}

```

Sposób kompilacji:

Nazwa kompilatora: g++

Wersja kompilatora: gcc version 8.5.0 20210514 (Red Hat 8.5.0-28) (GCC)

Kompilacja programu: g++ -fopenmp sequenceN350000.cpp -o sequence

Czas wykonania kodu sekwencyjnego 20 pomiarów:

Dla tablicy 5000:

- Najszybciej: iteracja 7 0.132988 s
- Najwolniej: iteracja 1 0.133573 s
- Średnia: 0.133283 s

Dla tablicy 350000:

- Najszybciej: iteracja 10/11 10.7914 min
- Najwolniej: iteracja 1 10.7932 min
- Średnia: 10.79224 min

Kod programu równoległego dla małej ilości danych:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <ctime>
```

```
#include <omp.h>
```

```
#include <algorithm>
```

```
void bubble_sort_parallel(std::vector<float>& arr) {
```

```
    int n = arr.size();
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (i % 2 == 0) {
```

```
#pragma omp parallel for default(none) shared(arr, n)
```

```
        for (int j = 1; j < n; j += 2) {
```

```
            if (arr[j - 1] > arr[j]) {
```

```
                double temp = arr[j - 1];
```

```

        arr[j - 1] = arr[j];

        arr[j] = temp;
    }
}
} else {
#pragma omp parallel for default(none) shared(arr, n)
    for (int j = 2; j < n; j += 2) {
        if (arr[j - 1] > arr[j]) {
            double temp = arr[j - 1];
            arr[j - 1] = arr[j];
            arr[j] = temp;
        }
    }
}
}

double random_fill(std::vector<float>,int size) {
    return static_cast<double>(rand()) / RAND_MAX * 100.0;
}

int main() {
    srand(time(NULL));

    std::vector<float> dane;

    int size = 3500;

```

```
dane.reserve(dane.size() + size);

for (int i = 0; i < size; i++) {
    dane.emplace_back(random_fill(dane, size));
}

double start = omp_get_wtime();
bubble_sort_parallel(dane);
double end = omp_get_wtime();

std::cout << "Time: " << (end - start) / 60.0 << " min\n";

return 0;
}
```

Sposób kompilacji:

Nazwa kompilatora: g++

Wersja kompilatora: gcc version 8.5.0 20210514 (Red Hat 8.5.0-28) (GCC)

Kompilacja programu: g++ -fopenmp bubbleSmall.cpp -o programSmall_omp

Kod programu równoległego dla dużej ilości danych:

```
#include <iostream>

#include <vector>

#include <ctime>

#include <omp.h>

#include <algorithm>
```

```

void bubble_sort_parallel(std::vector<float>& arr) {
    int n = arr.size();

    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            #pragma omp parallel for default(none) shared(arr, n)
            for (int j = 1; j < n; j += 2) {
                if (arr[j - 1] > arr[j]) {
                    double temp = arr[j - 1];
                    arr[j - 1] = arr[j];
                    arr[j] = temp;
                }
            }
        } else {
            #pragma omp parallel for default(none) shared(arr, n)
            for (int j = 2; j < n; j += 2) {
                if (arr[j - 1] > arr[j]) {
                    double temp = arr[j - 1];
                    arr[j - 1] = arr[j];
                    arr[j] = temp;
                }
            }
        }
    }
}

```

```

double random_fill(std::vector<float>, int size) {

```



```

        return static_cast<float>(rand()) / RAND_MAX * 100.0;
    }

int main() {
    srand(time(NULL));

    std::vector<float> dane;

    int size = 280000;

    dane.reserve(dane.size() + size);

    for (int i = 0; i < size; i++) {
        dane.emplace_back(random_fill(dane, size));
    }

    double start = omp_get_wtime();
    bubble_sort_parallel(dane);
    double end = omp_get_wtime();

    std::cout << "Time: " << (end - start) / 60.0 << " min\n";

    return 0;
}

```

Sposób kompilacji:

Nazwa kompilatora: g++

Wersja kompilatora: gcc version 8.5.0 20210514 (Red Hat 8.5.0-28) (GCC)

Kompilacja programu: g++ -fopenmp bubbleBig.cpp -o programBig_omp

Użyte elementy OpenMP w odniesieniu do algorytmu:

#pragma omp parallel for - Dyrektywa ta automatycznie tworzy zespół wątków i rozdziela pomiędzy nie iteracje pętli for. Ponieważ w danej fazie przetwarzane pary danych są od siebie niezależne, wątki mogą bezpiecznie wykonywać operacje zamiany (swap) współbieżnie na różnych fragmentach wektora arr, co drastycznie skraca czas sortowania.

omp_get_wtime(); - Funkcja służy do precyzyjnego pomiaru czasu rzeczywistego (*wall-clock time*) w sekundach, jaki upłynął od uruchomienia do zakończenia funkcji bubble_sort_parallel.

Treść skryptu kolejkowego:

```
#!/bin/bash -l

#SBATCH -N 1

#SBATCH --exclusive

#SBATCH -o plik_wyjsciwSmall.txt

echo "Rozpoczecie testow skalowalnosci OpenMP..."

echo "===== "

for threads in 1 2 4 8 12 16 20 24
do
    echo "Uruchamianie dla liczby watkow: $threads"

    export OMP_NUM_THREADS=$threads

    ./programSmall_omp

    echo "-----"
```

done

echo "Wszystkie testy zostały zakończone."

Specyfikacja komputera, na którym były przeprowadzone testy:

Procesor:

Model: Intel Xeon X5650

- Architektura: x86_64 (64-bit, 32-bit tryb zgodności)
- Liczba gniazd CPU: 2
- Rdzenie fizyczne: 12 (2 × 6 rdzeni)
- Wątki logiczne: 24 (HT: 2 wątki na rdzeń)
- Taktowanie:
 - bazowe / maksymalne: 2.67 GHz
 - minimalne: 1.6 GHz
- NUMA: 2 węzły
- Cache:
 - L1d: 32 KB / L1i: 32 KB
 - L2: 256 KB na rdzeń
 - L3: 12 MB
- Wirtualizacja: Intel VT-x

RAM:

- Pojemność całkowita: 46 GB
- Dostępna pamięć: ~43 GB
- Wykorzystana: ~2.2 GB
- Bufor/cache: ~4.0 GB
- Swap: brak (0 GB)

System operacyjny: AlmaLinux-8 (wersja 8.10)

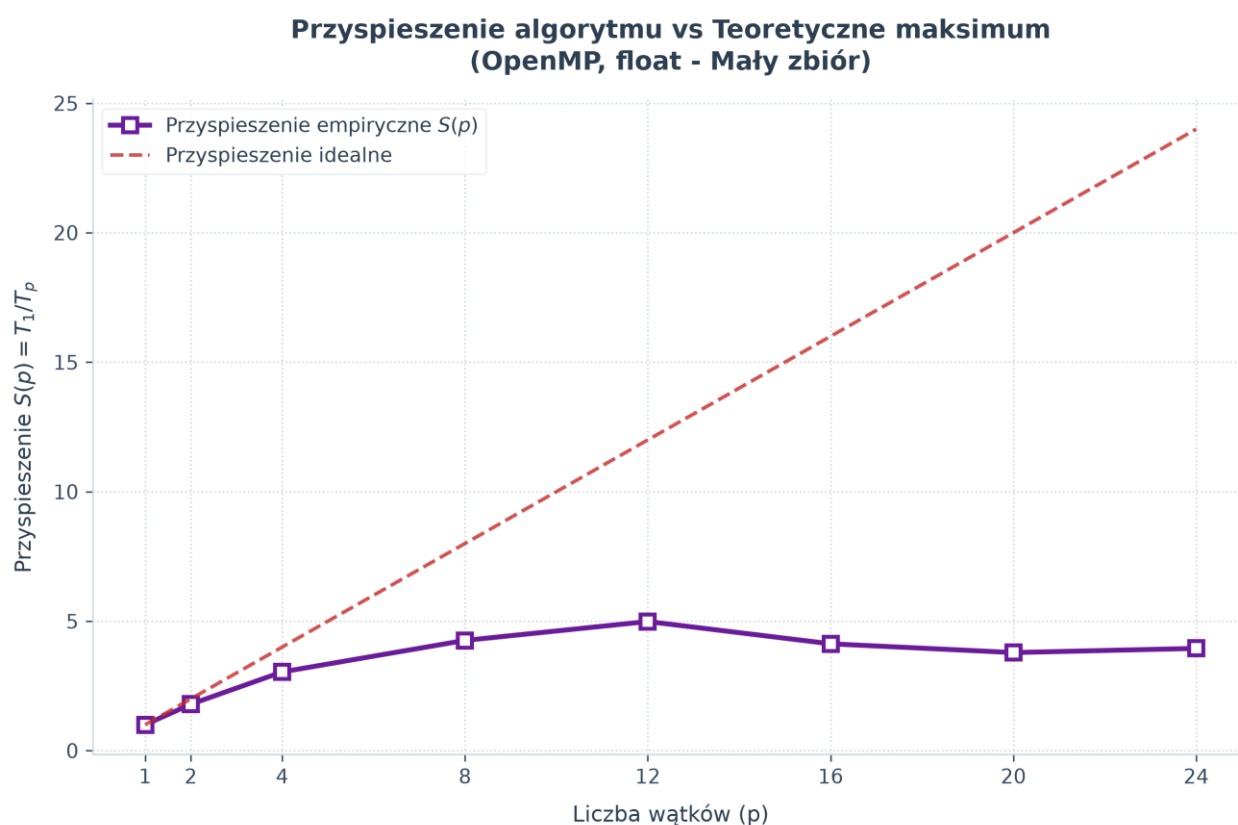
Tabele pokazujące czas wykonania dla implementacji równoległej algorytmu:

Rozmiar 3500	
Liczba rdzeni	Czas[min]
1	0.00197025
2	0.00109906
4	0.00064804
8	0.00046281
12	0.00039517
16	0.00047759
20	0.00052023
24	0.00049901

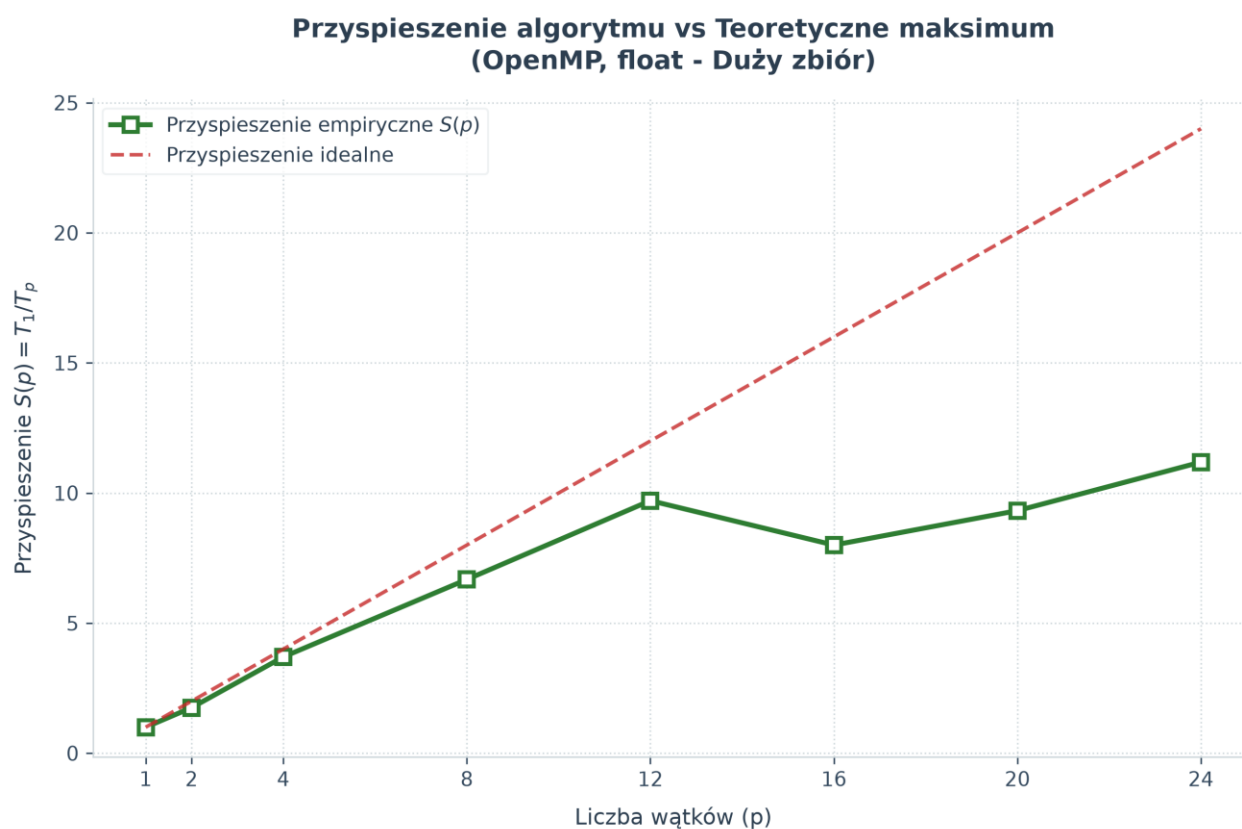
Rozmiar 280000	
Liczba rdzeni	Czas[min]
1	12.51240
2	7.15056
4	3.38088
8	1.87373
12	1.28912
16	1.56416
20	1.34218
24	1.11842

Wykresy przyspieszenia algorytmu równoległego:

Wykres 1



Wykres 2



Wnioski

Analiza została oparta na tablicach o rozmiarze 3500 i 280000 dla implementacji równoległej oraz 5000 i 350000 dla implementacji sekwencyjnej algorytmu Buble Sort. Doświadczenie polegało na zmierzeniu czasu wykonania programu sekwencyjnego i równoległego względem ilości wątków użytych do obliczeń.

Implementacja sekwencyjna dla małej ilości danych wykonywała się około 0.133283 s natomiast dla dużej ilości danych wykonanie programu zajmowało około 10.79224 min znacznie dłużej.

Dla implementacji równoległej przy małej ilości danych używanie wielu wątków prowadzi do zysku czasowego, jednak jest on nie zauważalny. Przy 1 wątku wykonanie programu zajęło 0.00197025 min (0,118s), gdzie przy 24 zajęło 0.00049901 min (0,030s). Używanie wielu wątków przy małej ilości danych jest zbędne natomiast przy dużej ilości danych zysk na czasie jest znaczny. Używając 1 wątku wykonanie programu zajmuje około 12 minut użycie 2 wątków skraca czas o około połowę do 7 minut podobna zmiana następuje przy użyciu 4 wątków natomiast od 8 wzwyż nie osiągamy aż tak dużego zysku na czasie nawet wykonanie programu trwa dłużej 12 wątków zajmuje 1.28 min a 20 1.34 min.

Poórwnując wykresy przyspieszenia dla implementacji równoległej można zaobserwować, że w obu przypadkach po 12 wątkach następuje spadek w wydajności dodawania coraz to większej ilości wątków zysk czasu staje się coraz mniejszy jak i przyspieszenie. Dla małej ilości danych powrót do podobnej efektywności nasypuje przy 24 wątkach, ale dochodzi do stagnacji przy dużej ilości danych przy 20 i powoli rośnie. Optymalną ilością wątków dla implementacji zdaje się 12 wątków program działa w miarę szybko i nie zużywa aż tak dużo zasobów.

Wynioskować można, że podczas implementowanie programu w oparciu o równoległość powinniśmy dobrać odpowiednią ilość wątków do ilości danych które będą przeliczane. Nadmierna ilość wątków może spowolnić prędkość wykonywania programu przez rozdzielanie zadań pomiędzy dostępne wątki. Wykorzystywanie wszystkich dostępnych zasobów obliczeniowych nie zawsze wpłynie dobrze na czas wykonanie programu.